Module 1 — Start with TIP

45-minute block · TIP4PATLIBS course material

> **How to read this document.** The running text addresses **you, the participant**. Boxes labelled **Trainer** are for whoever is **running the session**; boxes labelled **Trap** are traps that have caught people before.

Learning objective

I can open TIP, connect to PATSTAT, write and run my own first query — and set my environment up so that it survives a restart.

Prerequisites

- A TIP account with JupyterLab access. Nothing else.
- No Python, no SQL. Both are introduced here at the level you need them.

Sub-objectives

By the end you can:

1. **Say what persists and what does not** on TIP — which directory survives a restart, which files are overwritten on every start, and where your own settings therefore belong.
2. **Connect to PATSTAT** with `PatstatClient(env='PROD')` and read what came back as a DataFrame.
3. **Write a query by hand** — one applicant, one filter — and change it until it answers your question.
4. **Recognise the two counting mistakes** everyone makes on their first day: counting filings when you meant inventions, and counting inventors when you meant applicants.
5. **Connect Git over SSH** so your notebooks and results leave the machine.

Material

Folder	`1_startwithtip/`
Notebooks	`1_getting-started-with-tip.ipynb` (the environment) · `2_getting-started-with-patstat.ipynb` (your first queries)
Runs on	EPO TIP, PATSTAT PROD
Ships	with cleared outputs — you are meant to run it

> **Trainer.** Notebook 1 has eleven sections and notebook 2 has eleven queries. **You cannot do all of it in 45 minutes and you should not try.** The core path is set out in Phase 2: sections 1–5b and 9–11 of notebook 1, then Part 1 of notebook 2. Sections 6–8 (Claude Code) and Part 2 of notebook 2 (the institutional profile) are excellent self-study and are explicitly handed over as such in Phase 3.

Phase 1 · Introduction (≈ 7 min)

The question this module answers

> *You have been given a login to a machine you do not own, which will be reset without warning, and which holds the largest patent database in the world. Where do you put your work?*

That is not a rhetorical question. It is the one thing that goes wrong for almost everybody in their first week on TIP, and it goes wrong quietly: you install something, you customise your shell, you come back on Monday and it is gone. Not deleted — **overwritten**, by the startup scripts that rebuild the container every time it starts.

So this module is in two halves, and the first half is not about patents at all.

	Teaching and learning activity	Time
Opening	Trainer asks: *"you saved a file on TIP yesterday. Is it still there?"* — collect the guesses	2 min
Tension	Trainer names the real answer: it depends entirely on **which directory**, and nothing on the screen tells you which	2 min
Framing	Trainer sets out the two halves: *make the machine yours*, then *ask it a question*	3 min

> **Trainer.** The room splits reliably into people who assume everything persists and people who assume nothing does. Both are wrong, and the wrongness is productive. Do not resolve it — section 2 of the notebook resolves it, and it lands better when they have committed to a guess.

Phase 2 · Working through (≈ 28 min)

Step 1 — Learn the machine (10 min)

Open `1_getting-started-with-tip.ipynb` and run **sections 1 to 5b**. Each is one code cell that reports a fact about your own environment; nothing is installed and nothing is changed.

Section	The question it answers
1 · User identity & home directory	Who am I on this machine, and where is home?
2 · Filesystem mounts	Which directories persist, and which are rebuilt on every start?
3 · Training materials	Where does the course material come from, and why is it read-only?
4 · Startup scripts & dotfiles	Which of my own files get overwritten on every start?
5 · Python environment	Which Python am I running, and where does `epo.tipdata` live?
5b · Course dependencies	Do I have what the rest of the course needs?

Two results from these cells are worth writing down, because everything later depends on them:

> **Trap — Your home directory is `/home/jovyan` — via a symlink.** TIP uses `jovyan` as the base user; `/home/<your-username>` is a *symlink* to `/home/jovyan`. Both paths are the same directory, but they are **different strings**, and that breaks path arithmetic: `Path.home()` gives you the unresolved `/home/<username>` while `Path.cwd()` gives the resolved `/home/jovyan/...`, so `Path.cwd().relative_to(Path.home())` raises `ValueError`. Use `Path.home().resolve()`, or the `JUPYTER_SERVER_ROOT` environment variable.
>
> **Trap — Put shell customisations in `.bash_aliases`, never in `.bashrc`.** The startup scripts rewrite `.bashrc` on every container start. `.bash_aliases` is never touched. This one rule saves more frustration than anything else in this module.
>
> **Trainer.** Section 5 is where the *venv gotcha* lives: `epo.tipdata` is installed in the base conda environment. If a participant creates their own virtual environment and forgets to give it access, `from epo.tipdata.patstat import PatstatClient` fails with an import error that looks like a broken installation and is not. Name this before it happens.

Step 2 — Ask PATSTAT your first question (13 min)

Open `2_getting-started-with-patstat.ipynb`. Read the concepts cell — it is short — then run the setup cell and **Part 1, queries 1 to 3**.

Query	What it does	What you change
1 · Patents by applicant name	Finds every filing whose applicant name matches a pattern	`'%siemens%'` → your own company
2 · Filter by jurisdiction	Restricts to the offices you care about	the `AUTHORITIES` list
3 · Filings per jurisdiction and year	The same, broken out over time	the year window

Then run **query 4** — patent families for the same applicant — and compare the number it returns with query 1. It will be smaller. That difference is the whole lesson:

> **Rule of thumb.** Count `docdb_family_id` to count **inventions**. Count `appln_id` to count **filings**. One invention filed in eight countries is one family and eight filings, and the two numbers answer two different client questions.

And the second rule, which costs people entire analyses:

> **Trap — Always keep `applt_seq_nr > 0`.** Drop that condition and you are counting *inventors* alongside applicants. The query still runs, the numbers are plausible, and they are wrong.
>
> **Trainer.** Have the room say out loud which number they would give a client who asks *"how many patents does Siemens have?"* — and then ask what the client actually meant. There is no correct answer; there is only a stated one. That habit is the point.

Step 3 — Get your work off the machine (5 min)

Run **sections 9, 10 and 11** of notebook 1: connect Git over SSH, check your disk usage, and clone the training material into your own space.

> **Trap — Use an SSH key, not a Personal Access Token.** A fine-grained PAT is scoped to a single **resource owner**. A token owned by your personal account cannot reach an organisation's repositories — and the failure looks like a permissions bug on the repository, not like a problem with the token. The notebook generates a key and prints it for you to paste into GitHub.

Step	What you do	What you see	Time
1	Run sections 1–5b of notebook 1	A factual report on your own environment	10 min
2	Run Part 1, queries 1–4 of notebook 2	Your first result table — and the family/filing gap	13 min
3	Run sections 9–11 of notebook 1	A working SSH key and your own copy of the material	5 min

---

Phase 3 · Learning outcome (≈ 10 min)

What now exists

- A TIP environment that **survives a restart**, with your settings in the right file.
- One PATSTAT query you wrote yourself, answering a question you chose.
- An SSH key that lets you push results to your own repository.

Self-check

1. **You add an alias to `.bashrc` and restart the server. Is it still there?** *(No. Use `.bash_aliases`.)*
2. **A client asks how many patents their competitor holds. Which number do you give?** *(Whichever you can name: families = inventions, applications = filings. State which one you used.)*
3. **Your query returns twice as many "applicants" as you expected.** *(Check `applt_seq_nr > 0` — you are probably counting inventors too.)*
4. **`from epo.tipdata.patstat import PatstatClient` fails.** *(You are almost certainly in your own virtual environment without access to the base conda packages, not looking at a broken install.)*

Transfer to your own work

Take **one company you are regularly asked about** and run Part 1 for it: filings, jurisdictions, years, families. Save the notebook to your own repository. That is your baseline — every later module refines it.

> **Trainer.** Close by handing over what you skipped, explicitly and as an invitation, not as an apology:
> - **Sections 6–8 of notebook 1** install Claude Code persistently on TIP and give it a status line. Genuinely useful, entirely optional, and better done alone than in a room.
> - **Part 2 of notebook 2** profiles an institution end to end — name variants, portfolio size, timeline, one family under the microscope, filing strategy, technology fields. It is the natural evening exercise, and it is the direct precursor to module 3.

Where this leads

Next	Why
Module 2 — The Query Library	You have now written a query by hand. Module 2 is about *choosing* one that somebody has already made defensible — and knowing when to do which.
Module 3 — PATSTAT Explorer	Part 2 · Query 1 of notebook 2 shows you that one institution has many names. Module 3 is what you do about it.

Notes for the next revision

- Notebook 2 carries a stale example warning: it states that `han_name` shows *"TECHNISCHE UNIVERSITAT MUNCHEN"* for TU Berlin. Verify against the current PATSTAT edition before the workshop — the harmonisation may have been corrected, and a wrong warning teaches badly.
- Notebook 2 notes that the most recent two years are incomplete and names *2023–2024*. On PATSTAT Global Autumn 2025 the incomplete years are **2024–2025**. Update the text.
- Module 1 ships with **0 of 25 cells executed**, on purpose. Screenshots for the PDF and the slides therefore require a separate TIP run.
